LAB MANUAL

K.R. MANGALAM UNIVERSITY

SOET

(School of Engineering & Technology)

B.TECH CSE (CORE)

Database & Management System Lab (ENCS254)

Submitted by-

Akansha

Roll no - 2401010195

Submitted to-

Miss Mansi Kajal

LAB SHEET -01

Objective: To design ER diagrams and convert entities and relationships into relational tables using proper schema notation and to implement them using SQL queries.

PART A – COLLEGE DATABASE

Relational Schema
STUDENT (USN, SName, Address, Phone, Gender)
SEMSEC (SSID, Sem, Sec)
CLASS (USN, SSID)
SUBJECT (Subcode, Title, Sem, Credits)
IAMARKS (USN, Subcode, SSID, Test1, Test2, Test3, FinalIA)

Entities and Attributes

STUDENT is the entity **and** USN (Primary Key), SName, Address, Phone, Gender are attributes.

SEMSEC is the entity **and** SSID (Primary Key)**,** Sem**,** Sec are attributes.

SUBJECT is the entity **and** Subcode (Primary Key)**,** Title**,** Sem**,** Credits are attributes.

Relationships

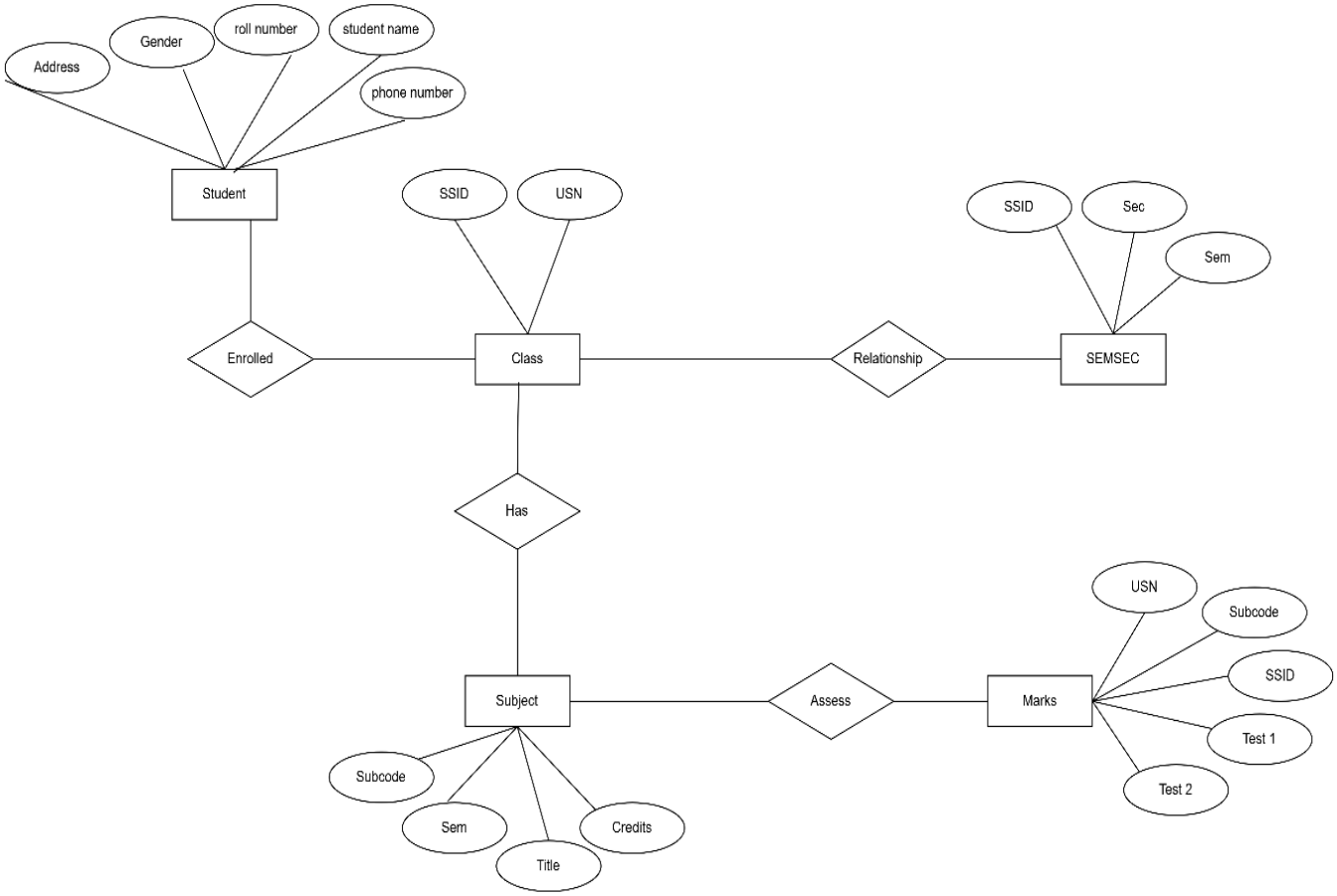
CLASS: Relationship between STUDENT and SEMSEC and it shows that a student belongs to a particular semester and section.
Type: Many-to-one (Many students belong to one semester and section)
IAMARKS: Relationship between STUDENT, SUBJECT, and SEMSEC and stores internal assessment marks of students
Attributes of IAMARKS are Test1, Test2, FinalIA
Type: Many-to-many relationship
Explanation:

STUDENT is an entity that stores student details such as USN, name, address, phone, and gender.
SEMSEC stores semester and section information using SSID, semester, and section.
SUBJECT contains subject details like subject code, title, semester, and credits.

The CLASS relationship shows that each student belongs to a particular semester and section. This is a many-to-one relationship between STUDENT and SEMSEC.

The IAMARKS relationship stores internal assessment marks of students for different subjects. It represents a many-to-many relationship between STUDENT and SUBJECT and

includes attributes like Test1, Test2, Test3, and FinalIA. This relationship is many-to-many because a student can take many subjects, and each subject can be taken by many students.



PART B – COMPANY DATABASE

ENTITIES:

1. EMPLOYEE**:** EMPLOYEE entity stores information about employees working in the company.
Attributes are :SSN (Primary Key), Name, Address, Sex, Salary, SuperSSN, DNo
SSN uniquely identifies each employee.
SuperSSN represents the supervisor of an employee (recursive relationship).
DNo is a foreign key which refers to the department in which the employee works.

2. DEPARTMENT: DEPARTMENT entity stores information about departments.
Attributes are: DNo (Primary Key), DName, MgrSSN, MgrStartDate
Each department is managed by one employee.
MgrSSN is a foreign key referring to EMPLOYEE.

3. PROJECT: PROJECT entity stores details of projects.
Attributes are: PNo (Primary Key), PName, PLocation, DNo

Each project is controlled by one department.
One department can control many projects.
4. DLOCATION: DLOCATION entity represents the different locations of a department.
Attributes: DNo, DLoc
A department can have multiple locations.
The primary key is the combination of DNo and DLoc.

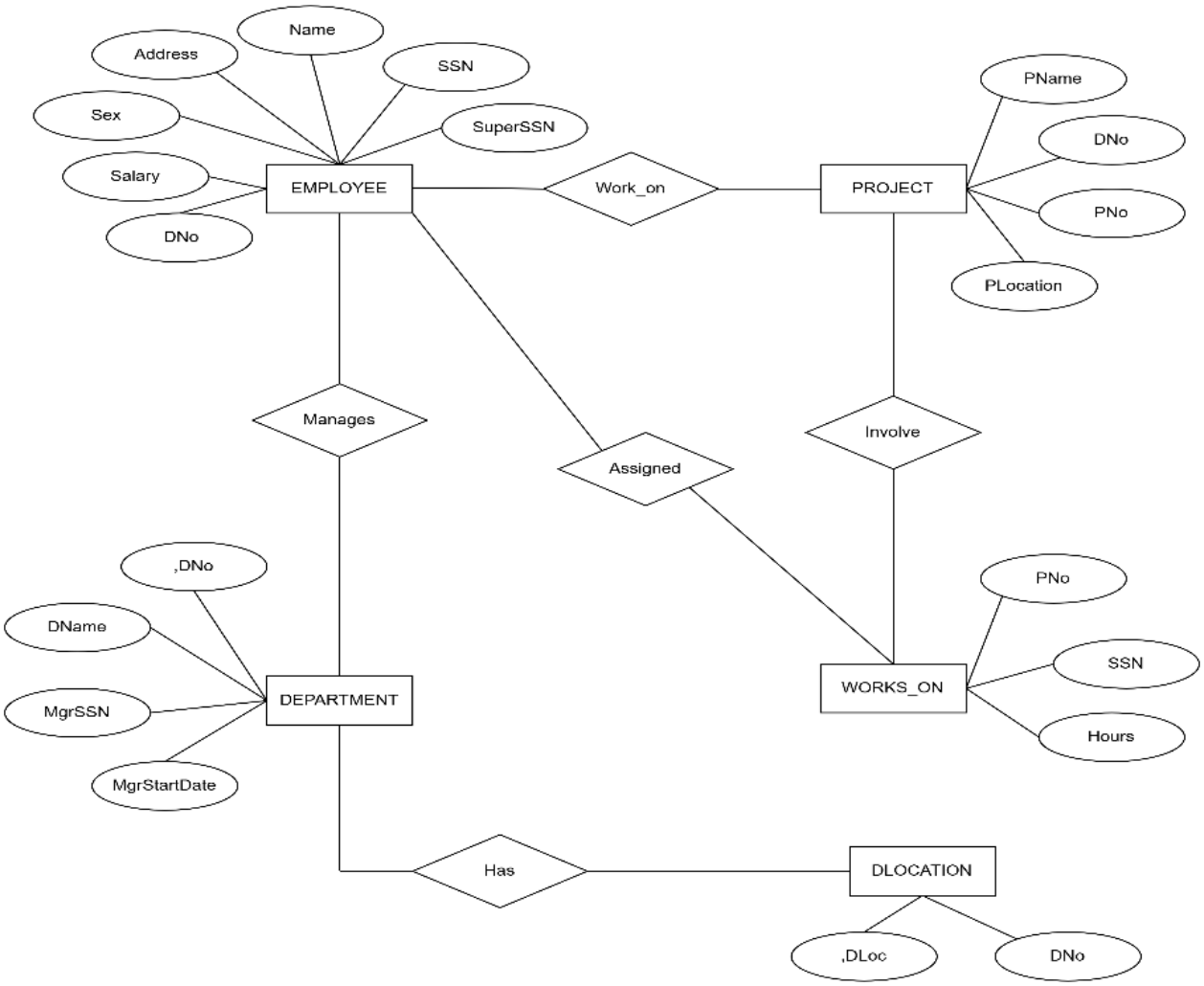
5. WORKS_ON: WORKS_ON entity represents the relationship between EMPLOYEE and PROJECT.
Attributes: SSN, PNo, Hours
This resolves the many-to-many relationship between employees and projects.
The primary key is the combination of SSN and PNo.

RELATIONSHIPS:

1. EMPLOYEE – WORKS_ON –PROJECT:
   An employee can work on many projects and a project can have many employees.
   This is a many-to-many relationship and is represented using WORKS_ON with attribute *Hours*.
2. EMPLOYEE – MANAGES – DEPARTMENT:
   An employee manages a department.
   Each department has one manager and an employee manages at most one department (1:1 relationship).
3. DEPARTMENT – ASSIGNED – PROJECT:
   A department controls many projects.
   Each project is controlled by only one department **(**1:N relationship).
4. DEPARTMENT – HAS – DLOCATION:
   A department can have multiple locations.
   This is a one-to-many (1:N) relationship.
5. EMPLOYEE – SUPERVISES – EMPLOYEE:
   An employee can supervise other employees.
   This is a recursive one-to-many (1:N) relationship.

EXPLANATION:

The ER diagram represents the COMPANY database.
EMPLOYEE stores employee details and each employee works in one department.
DEPARTMENT stores department details and is managed by an employee.
PROJECT stores project details and is controlled by a department.
WORKS_ON shows which employee works on which project and the number of hours.



PART C – CUSTOMER SALE SCHEMA

Database Schema for a customer-sale scenario:

Customer(Cust id : integer, cust_name: string)

Item(item_id: integer, item_name:string, price: integer)

Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

Solution: Entities and Attributes

1. CUSTOMER Entity The CUSTOMER entity stores information about customers who purchase products.

Attributes: • Cust_ID – Unique identification number of a customer (Primary Key) • Cust_Name – Name of the customer

2. SALE Entity The SALE entity represents transaction or billing information whenever a customer buys items.

Attributes: • Bill_No – Unique bill number (Primary Key)

• Bill_Date – Date on which purchase is made

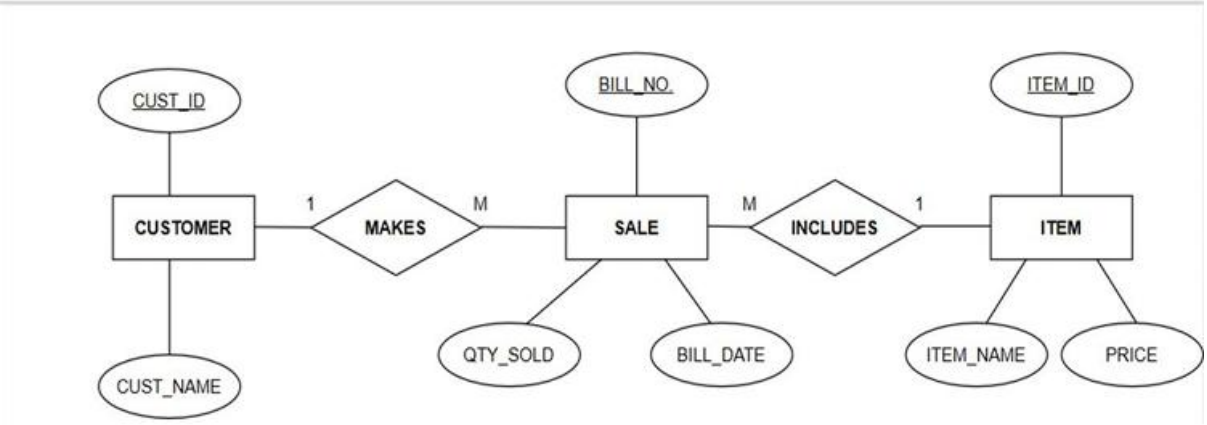
• Qty_Sold – Quantity of items purchased

3. ITEM Entity The ITEM entity contains details about products available for sale.

Attributes: • Item_ID – Unique product identification number (Primary Key)

• Item_Name – Name of the item

• Price – Cost of the item



DDL QUESTIONS

1. Write a statement to create all the above relations with appropriate PRIMARY KEY and FOREIGN KEY constraints.

Part A: COLLEGE DATABASE

We create database named DBMS_Akansha, and tables named STUDENT, SEMSEC, CLASS, SUBJECT, IAMARKS.

```
CREATE DATABASE DBMS_Akansha;
USE DBMS_Akansha;

CREATE TABLE STUDENT (
    USN VARCHAR(20) PRIMARY KEY,
    SName VARCHAR(50),
    Address VARCHAR(100),
    Phone VARCHAR(15),
    Gender VARCHAR(10)
);

CREATE TABLE SEMSEC (
    SSID VARCHAR(10) PRIMARY KEY,
    Sem INT,
    Sec VARCHAR(5)
);

CREATE TABLE CLASS (
    USN VARCHAR(20),
    SSID VARCHAR(10),
    PRIMARY KEY (USN, SSID),
    FOREIGN KEY (USN) REFERENCES STUDENT(USN),
    FOREIGN KEY (SSID) REFERENCES SEMSEC(SSID)
);
```

```
CREATE TABLE SUBJECT (
    Subcode VARCHAR(10) PRIMARY KEY,
    Title VARCHAR(50),
    Sem INT,
    Credits INT
);

CREATE TABLE IAMARKS (
    USN VARCHAR(20),
    Subcode VARCHAR(10),
    SSID VARCHAR(10),
    Test1 INT,
    Test2 INT,
    Test3 INT,
    FinalIA INT,
    PRIMARY KEY (USN, Subcode, SSID),
    FOREIGN KEY (USN) REFERENCES STUDENT(USN),
    FOREIGN KEY (Subcode) REFERENCES SUBJECT(Subcode),
    FOREIGN KEY (SSID) REFERENCES SEMSEC(SSID)
);
```

PART B: COMPANY DATABASE:

CODE:

```
CREATE TABLE Customer (
    Cust_id INT PRIMARY KEY,
    cust_name VARCHAR(50)
);

CREATE TABLE Item (
    item_id INT PRIMARY KEY,
    item_name VARCHAR(50),
    price INT NOT NULL
);

CREATE TABLE Sale (
    bill_no INT PRIMARY KEY,
    bill_date DATE,
    cust_id INT,
    item_id INT,
    qty_sold INT,
    FOREIGN KEY (cust_id) REFERENCES Customer(Cust_id),
    FOREIGN KEY (item_id) REFERENCES Item(item_id)
);
```

2. Add a new column Email to the STUDENT relation.

```
ALTER TABLE STUDENT
ADD Email VARCHAR(50);
SELECT*FROM STUDENT;
```

Result Grid | Filter Rows: | Edit: | Export/Import

USN	StudentName	Address	Phone	Gender	Email
NULL	NULL	NULL	NULL	NULL	NULL

3. Apply NOT NULL constraints to Salary and Price attributes.

- ALTER TABLE is used to modify an existing table.
- MODIFY changes the definition of a column.
- NOT NULL ensures that the column cannot store NULL values.

```
ALTER TABLE EMPLOYEE
MODIFY Salary INT NOT NULL;

ALTER TABLE Item
MODIFY price INT NOT NULL;
```

4.Rename attribute SName to StudentName in STUDENT relation.

```
ALTER TABLE STUDENT
RENAME COLUMN SName TO StudentName;
SELECT*FROM STUDENT;
```

Result Grid | Filter Rows: | Edit:

USN	StudentName	Address	Phone	Gender	Email
NULL	NULL	NULL	NULL	NULL	NULL

5.Drop the DLOCATION relation.

```
DROP TABLE DLOCATION;
```

DML QUESTION:

1. **Insert at least 10 tuples into CUSTOMER, ITEM, and SALE relations.**

```
INSERT INTO Customer VALUES
(1,'Rahul'),(2,'Amit'),
(3,'Sneha'),(4,'Priya'),
(5,'Karan'),(6,'Anjali'),
(7,'Rohit'),(8,'Meena'),
(9,'Vikas'),(10,'Neha');
SELECT*FROM Customer;
```

Result Grid | Filter Rows:

Cust_id	cust_name
1	Rahul
2	Amit
3	Sneha
4	Priya
5	Karan
6	Anjali
7	Rohit
8	Meena
9	Vikas
10	Neha
NULL	NULL

```sql
INSERT INTO Item VALUES
(101,'Pen',50),
(102,'Book',250),
(103,'Bag',800),
(104,'Pencil',20),
(105,'Shoes',1500),
(106,'Bottle',200),
(107,'Notebook',100),
(108,'Watch',2000),
(109,'Charger',500),
(110,'Headphones',1200);
SELECT*FROM Item;
```

Result Grid | Filter Rows:

item_id	item_name	price
101	Pen	50
102	Book	250
103	Bag	800
104	Pencil	20
105	Shoes	1500
106	Bottle	200
107	Notebook	100
108	Watch	2000
109	Charger	500
110	Headphones	1200
NULL	NULL	NULL

```sql
INSERT INTO Sale VALUES
(1,CURDATE(),1,101,2),
(2,CURDATE(),2,102,1),
(3,CURDATE(),3,103,1),
(4,CURDATE(),4,104,5),
(5,CURDATE(),5,105,1),
(6,CURDATE(),6,106,3),
(7,CURDATE(),7,107,4),
(8,CURDATE(),8,108,1),
(9,CURDATE(),9,109,2),
(10,CURDATE(),5,110,1);
SELECT*FROM Sale;
```

Result Grid | Filter Rows: | Edi

bill_no	bill_date	cust_id	item_id	qty_sold
1	2026-03-01	1	101	2
2	2026-03-01	2	102	1
3	2026-03-01	3	103	1
4	2026-03-01	4	104	5
5	2026-03-01	5	105	1
6	2026-03-01	6	106	3
7	2026-03-01	7	107	4
8	2026-03-01	8	108	1
9	2026-03-01	9	109	2
10	2026-03-01	5	110	1
NULL	NULL	NULL	NULL	NULL

2. **List all bills of the current date with customer names and item names.**

```
SELECT s.bill_no, c.cust_name, i.item_name
FROM Sale s
JOIN Customer c ON s.cust_id = c.Cust_id
JOIN Item i ON s.item_id = i.item_id
WHERE s.bill_date = CURDATE();
```

Result Grid | Filter Rows: | Export:

bill_no	cust_name	item_name
1	Rahul	Pen
2	Amit	Book
3	Sneha	Bag
4	Priya	Pencil
5	Karan	Shoes
6	Anjali	Bottle
7	Rohit	Notebook
8	Meena	Watch
9	Vikas	Charger
10	Karan	Headphones

3. **Display total bill amount using quantity and price.**

```
SELECT s.bill_no, (s.qty_sold * i.price)
AS Total_Amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
```

Result Grid | Filter Rows: | Export:

bill_no	Total_Amount
1	100
2	250
3	800
4	100
5	1500
6	600
7	400
8	2000
9	1000
10	1200

4. **List customers who purchased products priced above 200.**

```
SELECT DISTINCT c.cust_name
FROM Sale s
JOIN Customer c ON s.cust_id = c.Cust_id
JOIN Item i ON s.item_id = i.item_id
WHERE i.price > 200;
```

Result Grid | Filter Rows: | Export:

cust_name
Amit
Sneha
Karan
Meena
Vikas

5. **Count the number of products purchased by each customer.**

```
SELECT c.cust_name, COUNT(s.item_id) AS Total_Products
FROM Sale s
JOIN Customer c ON s.cust_id = c.Cust_id
GROUP BY c.cust_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cust_name	Total_Products
Rahul	1
Amit	1
Sneha	1
Priya	1
Karan	2
Anjali	1
Rohit	1
Meena	1
Vikas	1

6. **List products purchased by customer having Cust_id = 5**

```
SELECT i.item_name
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.cust_id = 5;
```

Result Grid | Filter Rows: | Export:

item_name
Shoes
Headphones

7. **Create a VIEW showing bill_no, bill_dates, cust_id, price, qty_sold, amount.**

```
CREATE VIEW Bill_Details AS
SELECT s.bill_no, s.bill_date, s.cust_id, s.item_id,
       i.price, s.qty_sold,
       (i.price * s.qty_sold) AS Amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
SELECT*FROM Bill_Details;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conten

bill_no	bill_date	cust_id	item_id	price	qty_sold	Amount
1	2026-03-01	1	101	50	2	100
2	2026-03-01	2	102	250	1	250
3	2026-03-01	3	103	800	1	800
4	2026-03-01	4	104	20	5	100
5	2026-03-01	5	105	1500	1	1500
6	2026-03-01	6	106	200	3	600
7	2026-03-01	7	107	100	4	400
8	2026-03-01	8	108	2000	1	2000
9	2026-03-01	9	109	500	2	1000
10	2026-03-01	5	110	1200	1	1200

8. **Create a VIEW showing daily sales date-wise for the last one week .**

```
CREATE VIEW Weekly_Sales AS
SELECT bill_date, SUM(qty_sold * i.price) AS Daily_Total
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE bill_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY bill_date;
SELECT*FROM Weekly_Sales;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_date	Daily_Total
2026-03-01	7950